

มหาวิทยาลัยมหิดล
Mahidol University



Mahidol
University

ITCS215
Introduction to Interactive Multimedia Technology

Semester 1/2026

Pre-processing and Colors

Assoc. Prof. Dr. Worapan Kusakunniran

Bitmapped Images

- Also known as ***raster graphics***
  - Record a value for ***every pixels*** in the image

- For example: RGB color model
  - Each pixel contains the value of Red, Green, and Blue colors
  - 8 bits for each color value in each pixel

Pixel Information

- One channel
  - Grey-scale
  - E.g., 8 bits for each pixel



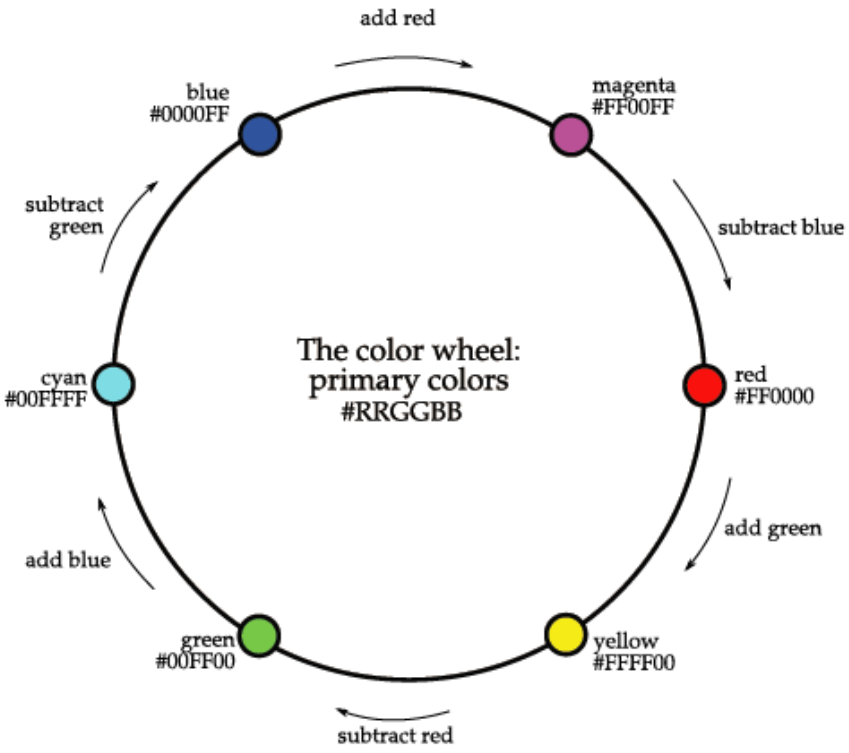
- Three channels
  - RGB
  - HSV
  - CMYK
  - E.g., 24 bits for each pixel (8 + 8 + 8 pixels)



https://www.tutorialspoint.com/java_dip/grayscale_conversion.htm

RGB Color Model

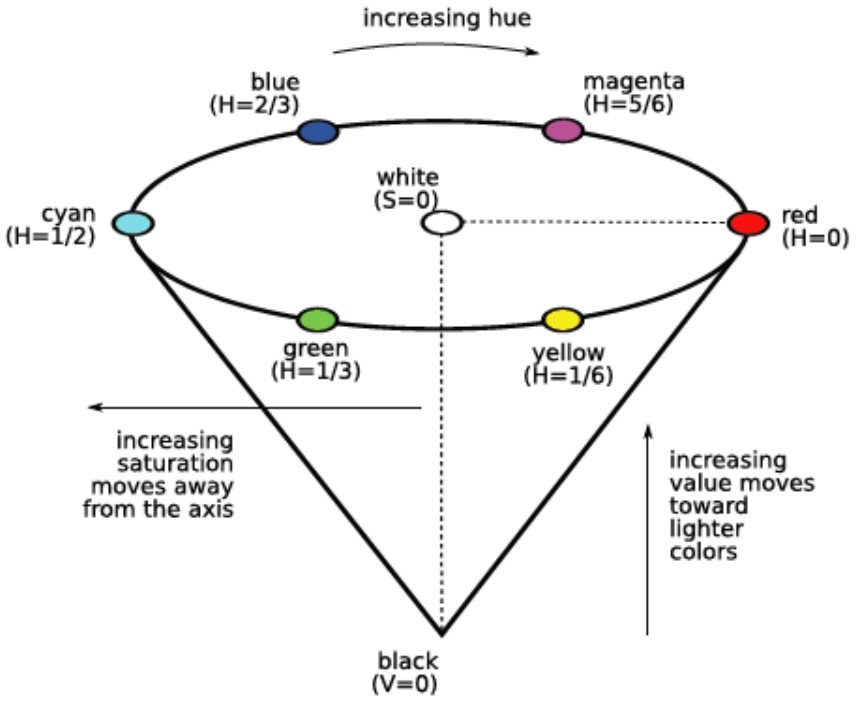
- It considers only the ***pure colors***
- Its ***primary colors*** are red (R), green (G) and blue (B)
- Any pure color is a mixture of red, green and blue
  - E.g., start with ***red*** light, add some ***green*** light → color shifts toward ***yellow***
  - E.g., start with ***yellow*** light, reduce some ***red*** light → color shifts toward ***green***



J. W. Shipman, "Introduction to color theory"

HSV Color Model

- The hue (H): which pure color it resembles
- The saturation (S): how white the color is
  - E.g., the pure red color is fully saturated → a saturation =1
  - E.g., the white color has a saturation = 0
  - E.g., the tints of the red color have a saturation < 1
- The value (V): the lightness of the color describing how dark the color is



J. W. Shipman, “Introduction to color theory”

CMY Color Model

- Its main colors are cyan (C), magenta (M) and yellow (Y)
- CMY is an opposite of RGB
  - Red is opposite cyan (C = 1 – R, if each value is between 0 and 1)
  - Green is opposite magenta (M = 1 – G, if each value is between 0 and 1)
  - Blue is opposite yellow (Y = 1 – B, if each value is between 0 and 1)
- It is a ***subtractive primary color*** model
  - Start with the white light, we can filter out various amounts of CMY to produce any color
  - Colored inks in paper act to filter out reflected colors

J. W. Shipman, “Introduction to color theory”

Enhancing Method

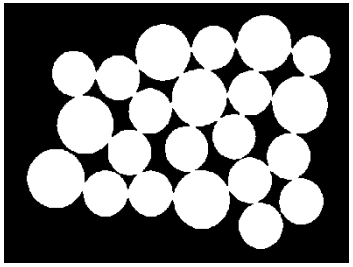
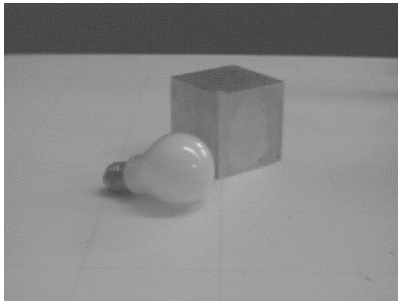
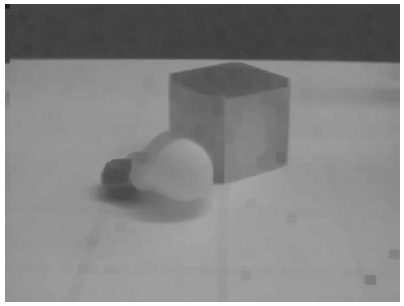
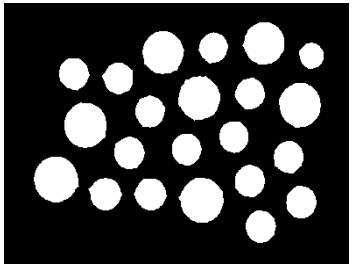
Morphological Operators

- Structuring Element
- Erosion
- Dilation
- Opening
- Closing

Morphological Operators

- Erosion
  - **shrinks** the connected sets of 1s of a binary image



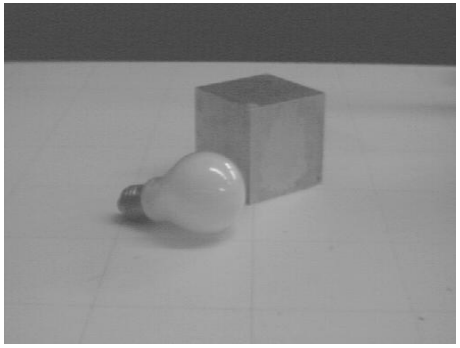
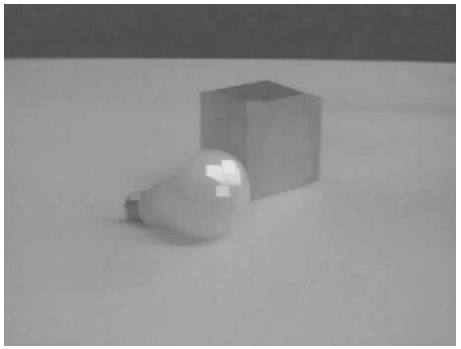





https://docs.opencv.org/2.4/doc/tutorials/imgproc/erosion_dilatation/erosion_dilatation.html

Morphological Operators

- Dilation
  - **expands** the connected sets of 1s of a binary image

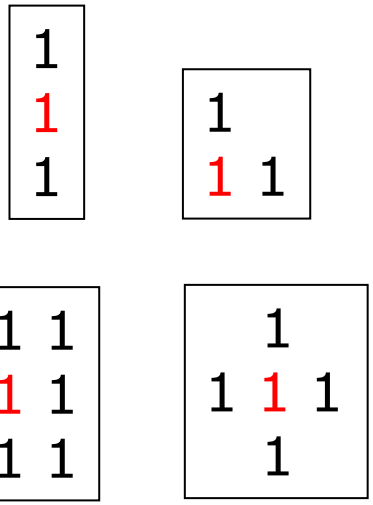


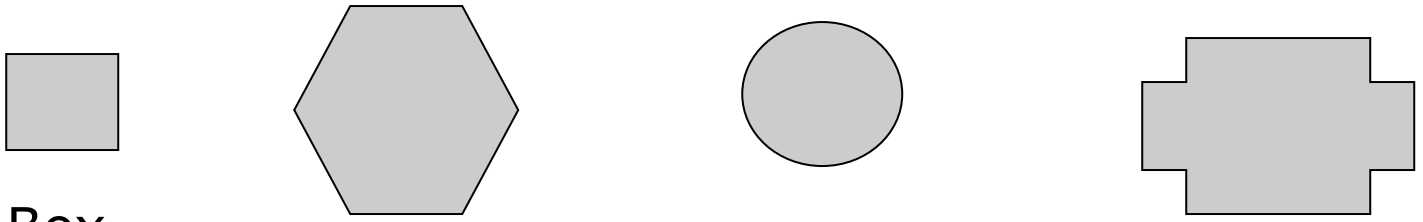



https://docs.opencv.org/2.4/doc/tutorials/imgproc/erosion_dilatation/erosion_dilatation.html

Morphological Operators

- Structuring Element
  - **a shape mask** used in the basic morphological operations
    - be any shape and size
    - has an origin



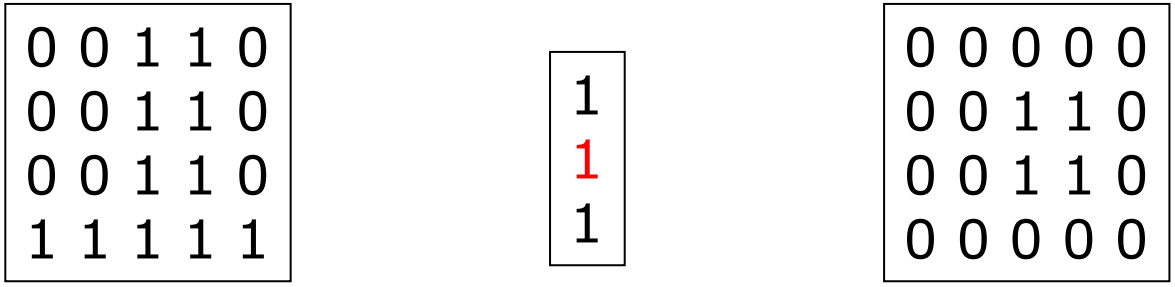


Box (width, height)

disk (diameter)

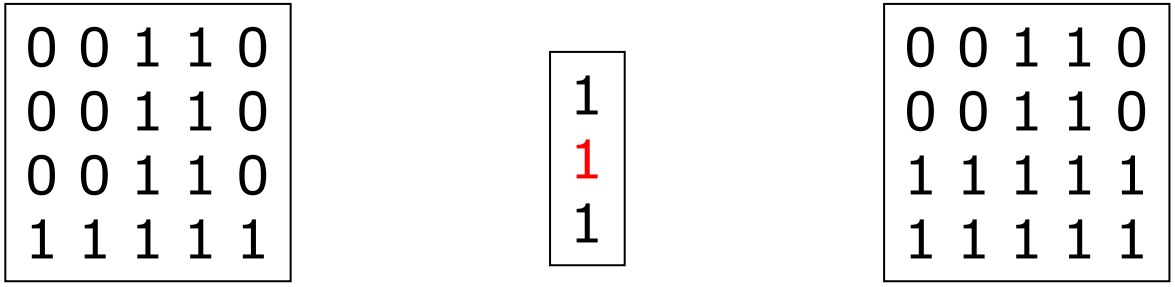
Morphological Operators

- Erosion



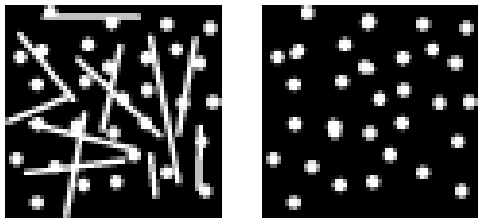
Morphological Operators

- Dilation



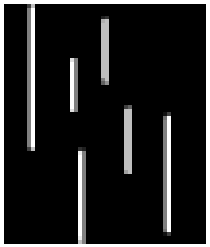
Morphological Operators

- Opening
  - compound operation of **erosion** followed by **dilation** (with the same structuring element)

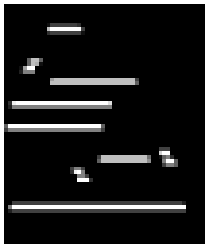


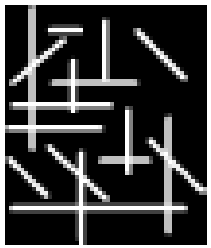
Opening with a **disk-shaped** structuring element **11** pixels in diameter

opening with a **3×9** vertically oriented structuring element



opening with a **9×3** horizontally oriented structuring element

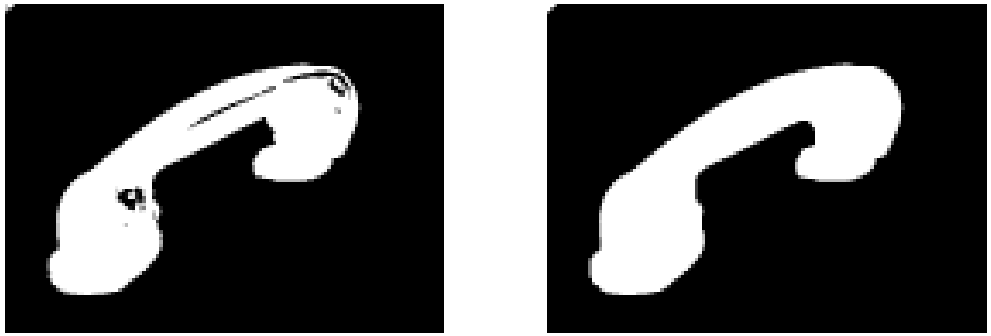




https://homepages.inf.ed.ac.uk/rbf/HIPR2/open.htm

Morphological Operators

- Closing
  - compound operation of **dilation** followed by **erosion** (with the same structuring element)



https://homepages.inf.ed.ac.uk/rbf/HIPR2/close.htm#variants

Act 3.1

- Input

Section 1

1	1	1	0	0
1	0	1	0	0
1	0	1	0	0
1	1	1	1	1

Section 2

0	1	1	0	0
0	0	1	1	1
0	0	1	0	1
0	1	1	1	1

Section 3

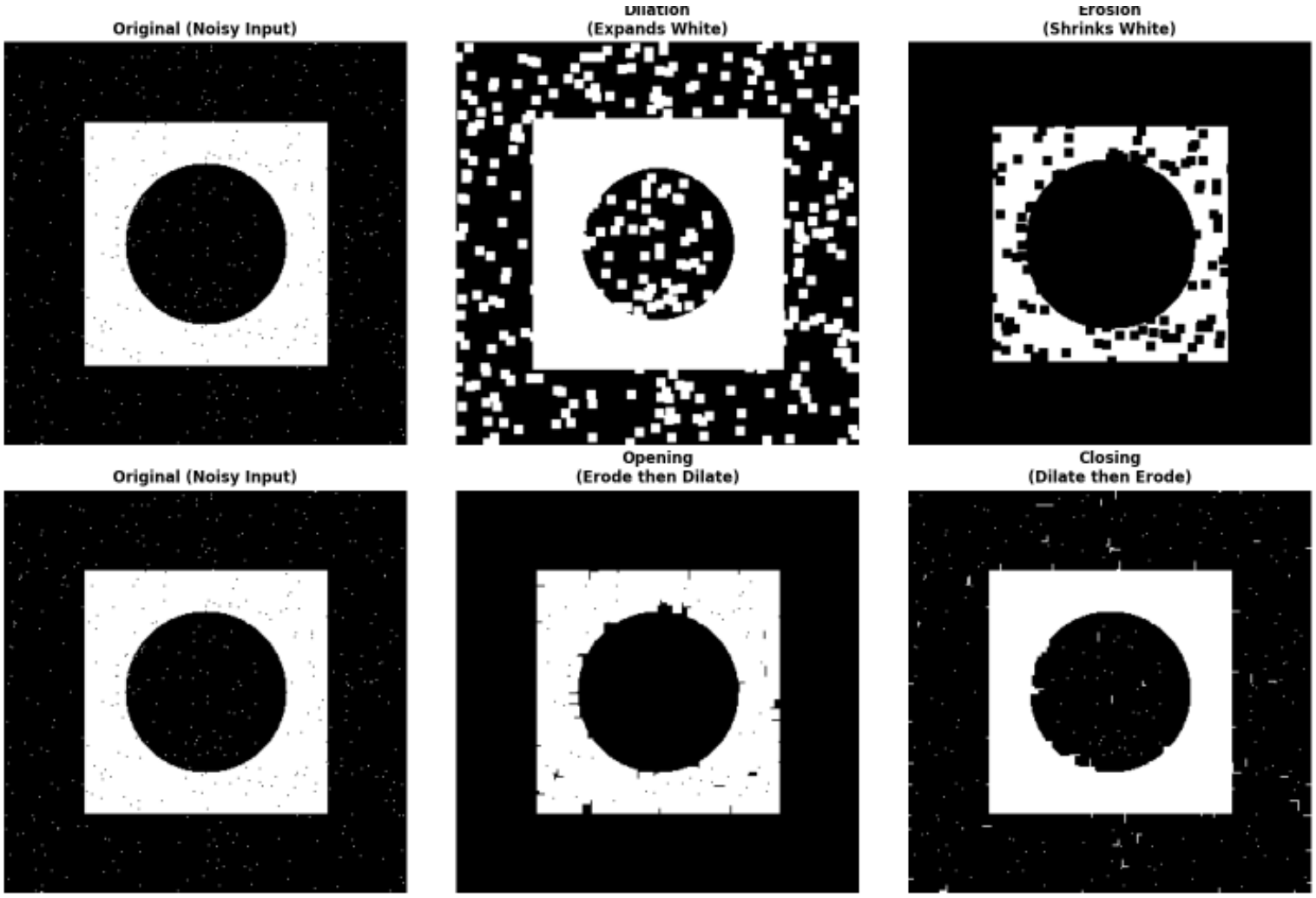
0	0	0	0	1
0	1	1	1	1
0	1	0	1	1
0	1	1	1	1

- S.E.

1	1	1
1	1	1
1	1	1

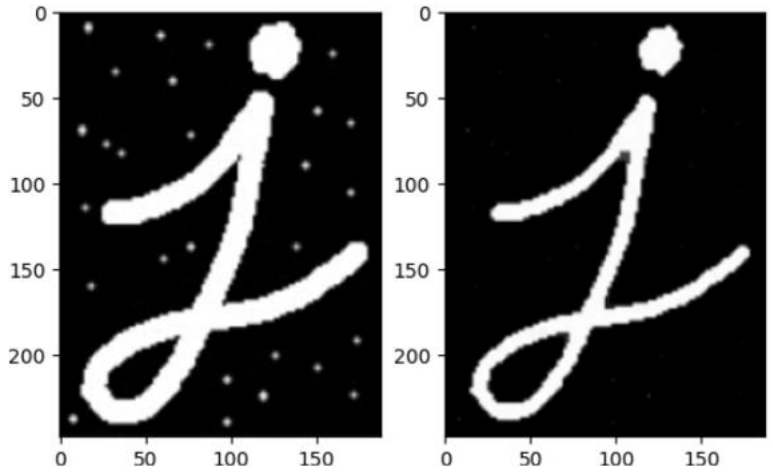
- Perform
  - Closing operation
  - Padding with 0

Act 3.2

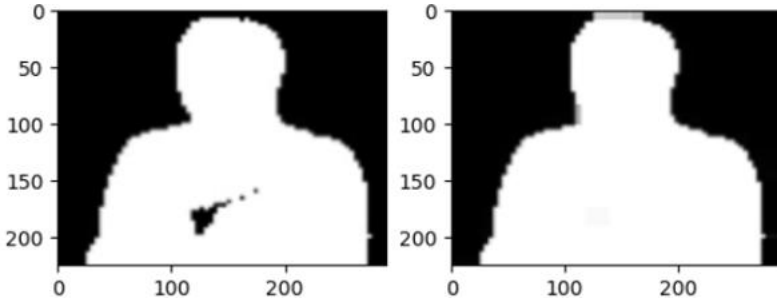

Original (Noisy Input)
Dilation
(Expands White)
Erosion
(Shrinks White)
Original (Noisy Input)
Opening
(Erode then Dilate)
Closing
(Dilate then Erode)

Act 3.2

- Write a python code to implement a chosen method of morphological operations to get the desired output
- 3.2.1



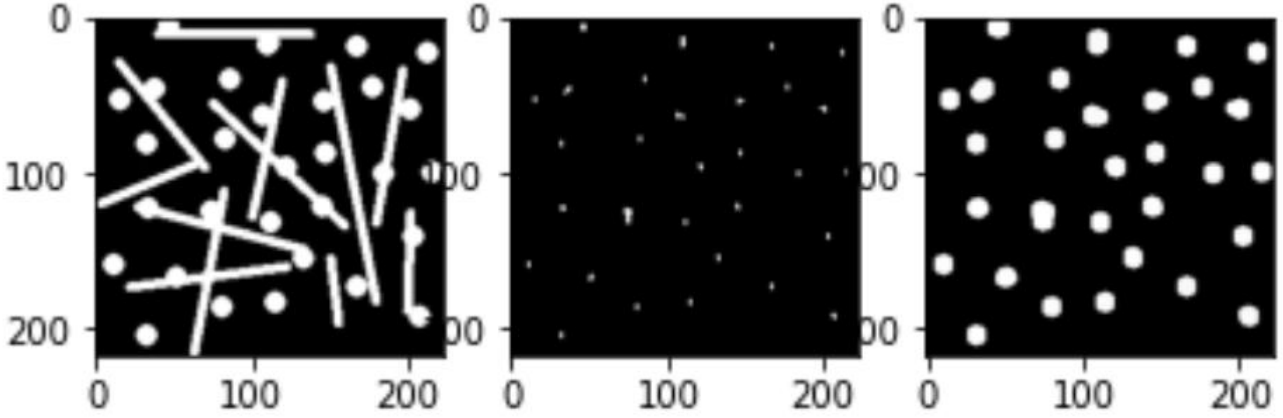
- 3.2.2



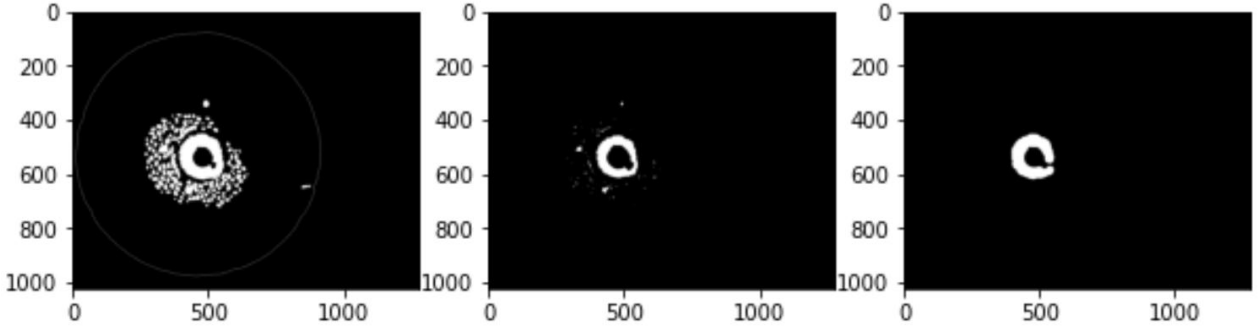

```
kernel_size = (9, 9)ellipse_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, kernel_size)
img_closed = cv2.morphologyEx(img, cv2.MORPH_CLOSE, ellipse_kernel)
```

Act 3.2

- Write a python code to implement a chosen method of morphological operations to get the desired output
- 3.2.3

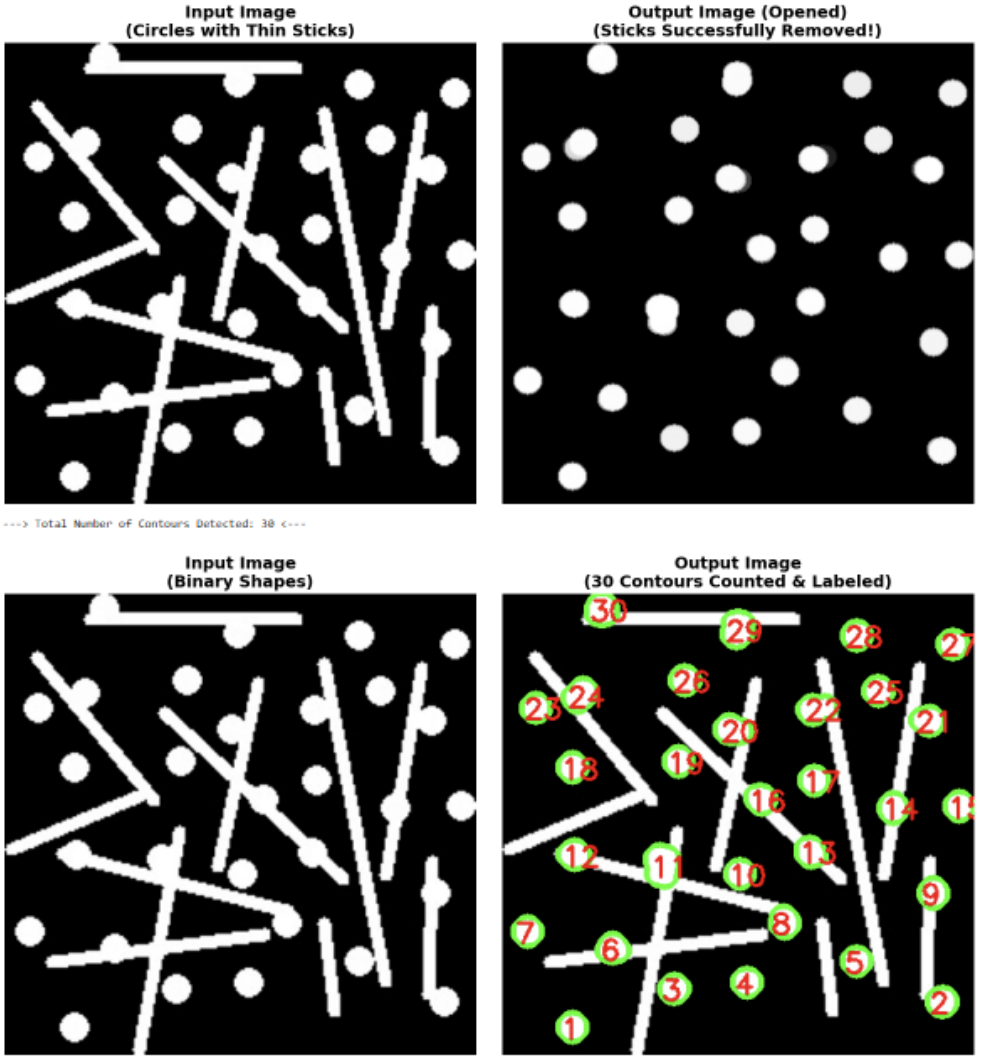


- 3.2.4



Act 3.2

- Write a python code to implement a chosen method of morphological operations to get the desired output
- 3.2.5 DO the counting of circles
  - Do “opening” for removing sticks before the dot counting



Histogram equalization

- Histogram
  - Grayscale, L = 8 (3 bits)

0	1	1	2	4
2	1	0	0	2
5	2	0	0	4
1	1	2	4	1

Image

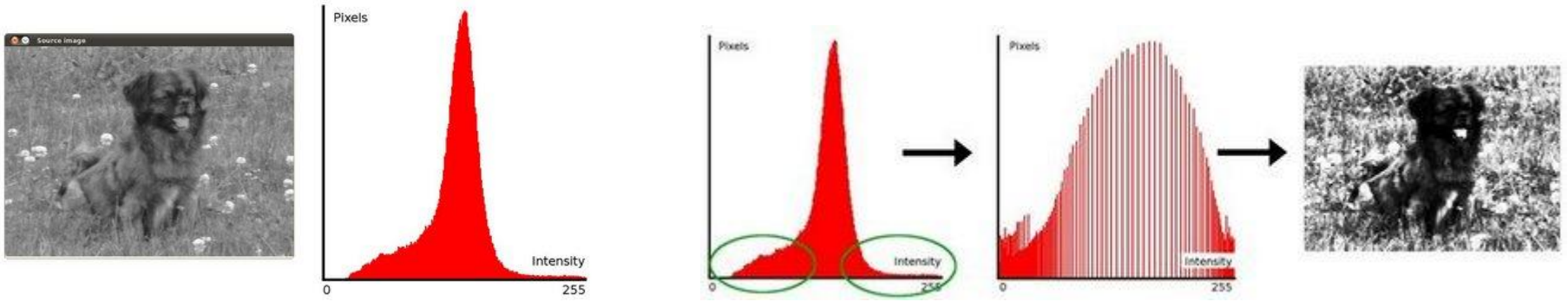


Histogram

Histogram equalization

- **Improves the contrast** in an image, in order to stretch out the intensity range.

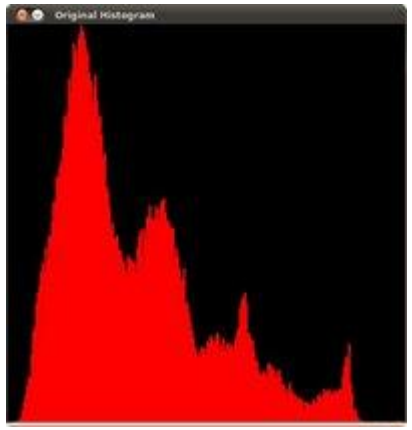
- Mapping one distribution (the given histogram) to another distribution (a wider and more **uniform distribution** of intensity values).



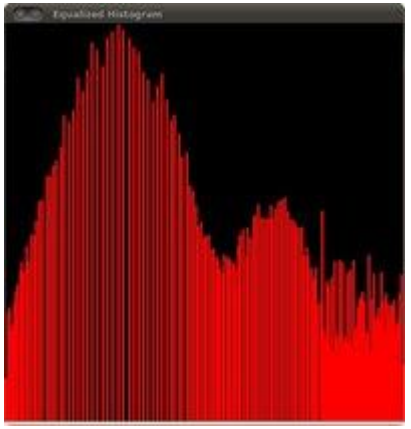
https://docs.opencv.org/2.4/doc/tutorials/imgproc/histograms/histogram_equalization/histogram_equalization.html

Histogram equalization



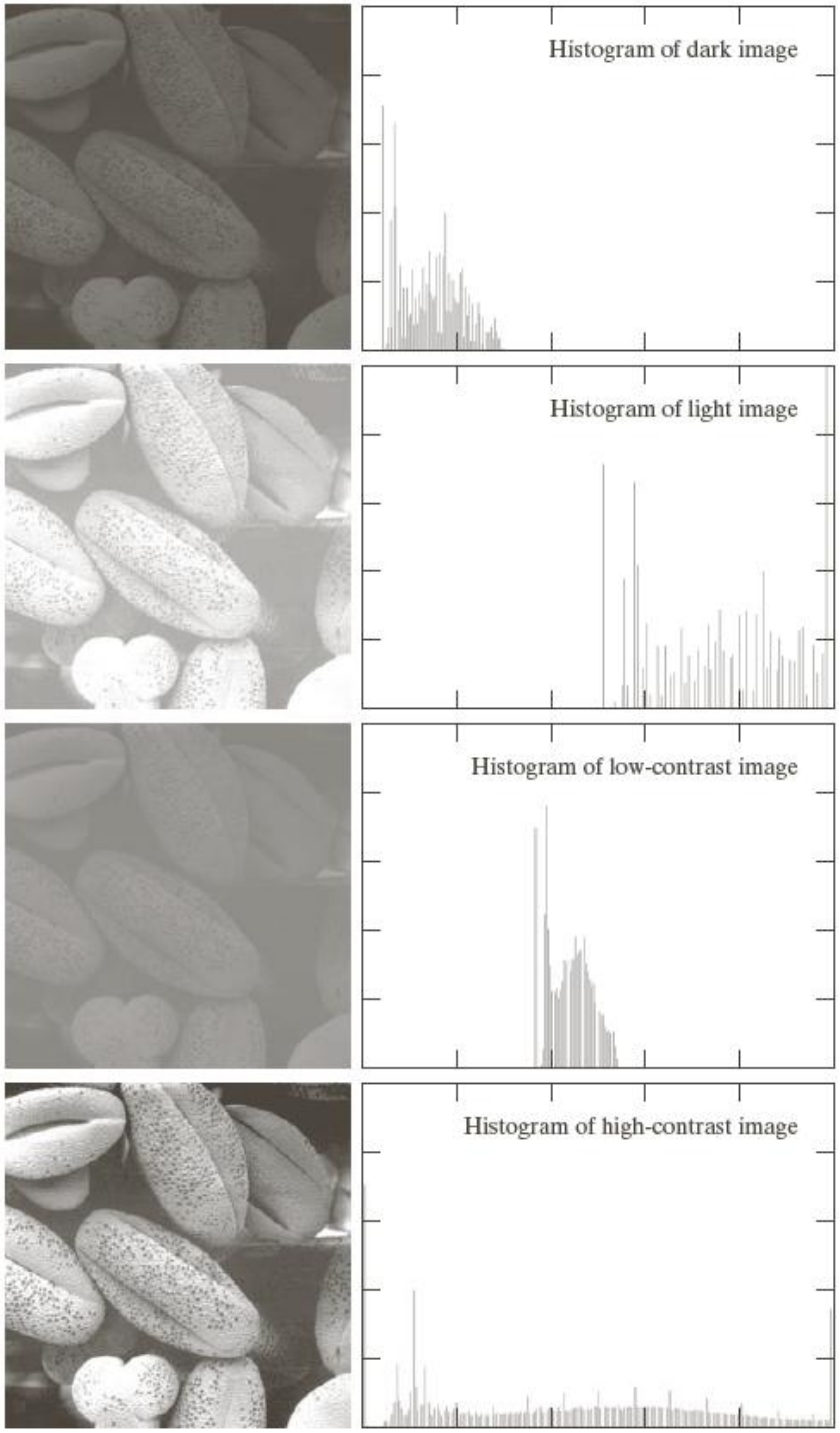






https://docs.opencv.org/2.4/doc/tutorials/imgproc/histograms/histogram_equalization/histogram_equalization.html

Histogram equalization

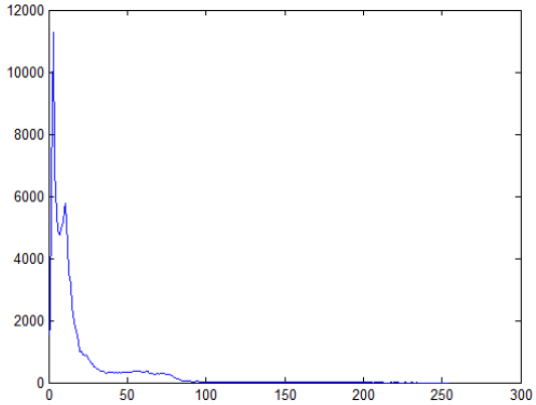


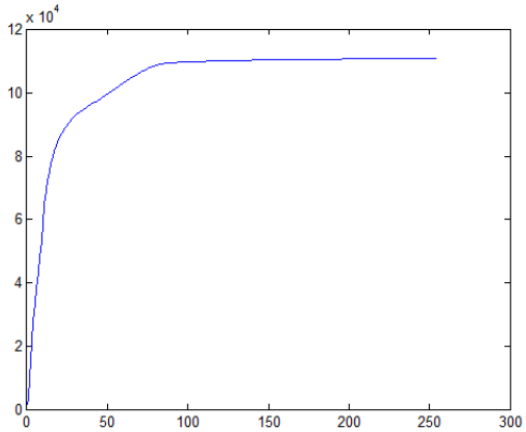
Histogram equalization

- Create the histogram for the image
- Calculate the cumulative distribution function histogram

$$\mathrm{cdf}(x) = \sum_{j=1}^{x} h(j)$$

- Calculate the new values through the general histogram equalization formula
- Assign new values for each gray value in the image





http://cache.freescale.com/files/dsp/doc/app_note/AN4318.pdf

Histogram equalization

- Calculate the new values through the general histogram equalization formula

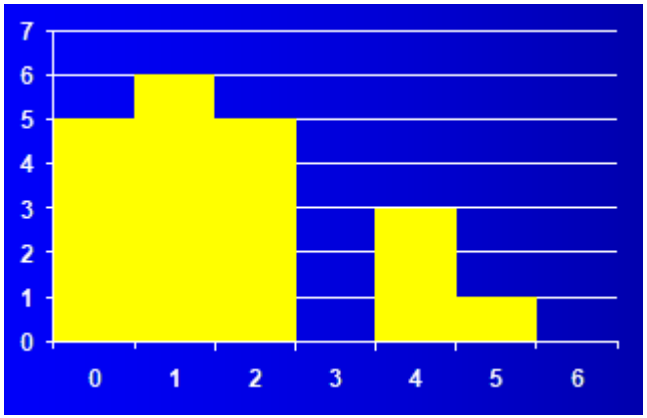
$$eh(i) = \text{round}\left(\frac{cdf(1) - cdf_{min}}{MxN - cdf_{min}} * (L - 1)\right)$$

where cdfmin is the minimum value of the cumulative distribution function, M x N are the image's number of columns and rows, and L is the number of gray levels used (in most cases 256). This formula is implemented in this code:

http://cache.freescale.com/files/dsp/doc/app_note/AN4318.pdf

Act 3.3

- Perform the histogram equalization on the following histogram and draw up the updated histogram
  - Grayscale, L = 8 (3 bits)



0	1	1	2	4
2	1	0	0	2
5	2	0	0	4
1	1	2	4	1

Image

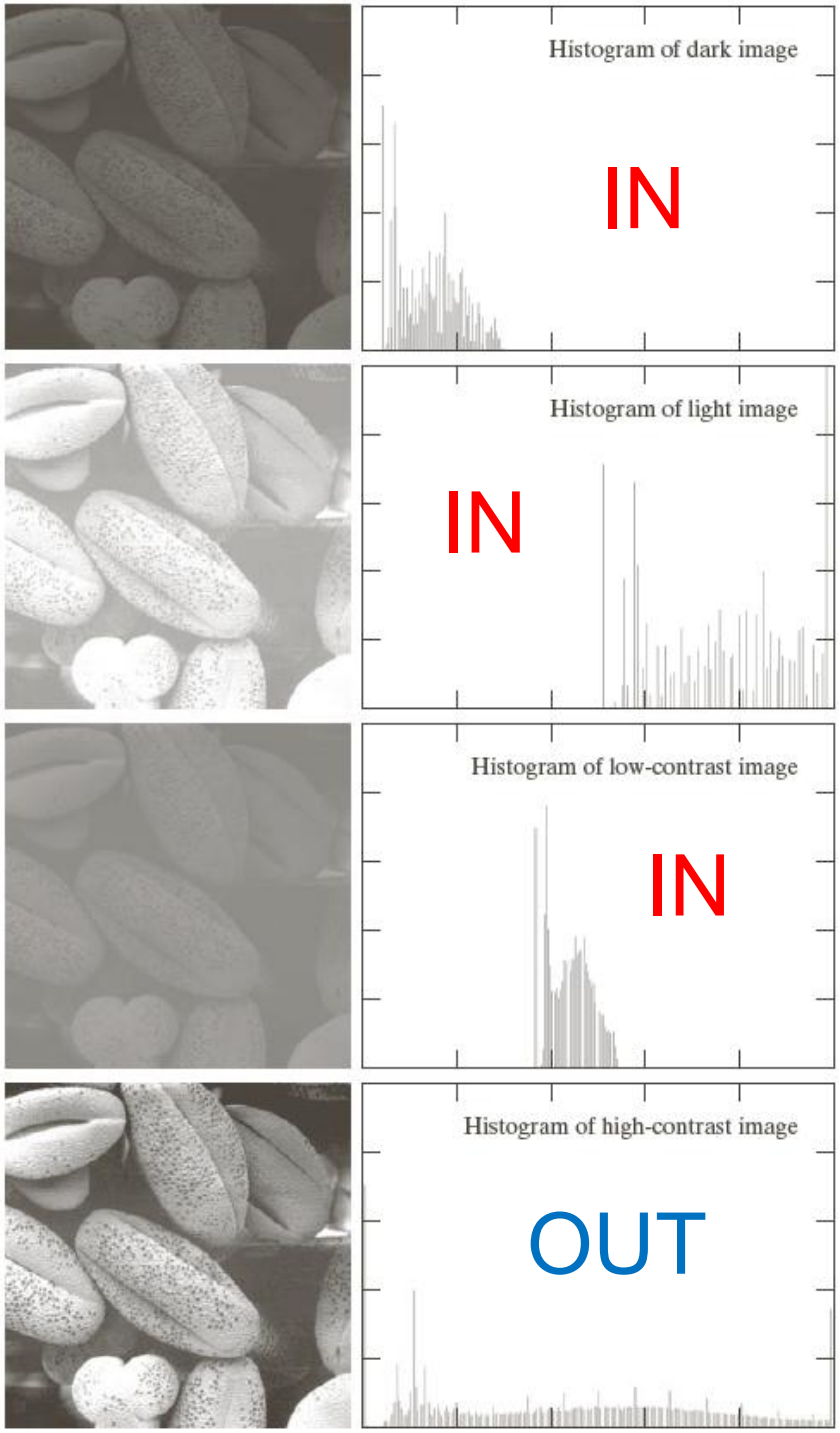
$$eh(i) = \text{round}\left(\frac{cdf(1) - cdf_{min}}{MxN - cdf_{min}} * (L-1)\right)$$

where cdfmin is the minimum value of the cumulative distribution function, M x N are the image's number of columns and rows, and L is the number of gray levels used (in most cases 256). This formula is implemented in this code:

http://cache.freescale.com/files/dsp/doc/app_note/AN4318.pdf

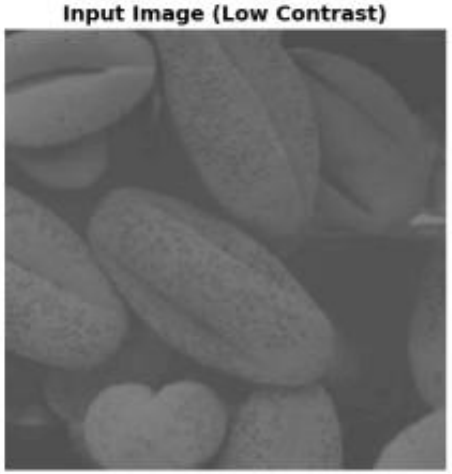
Act 3.4

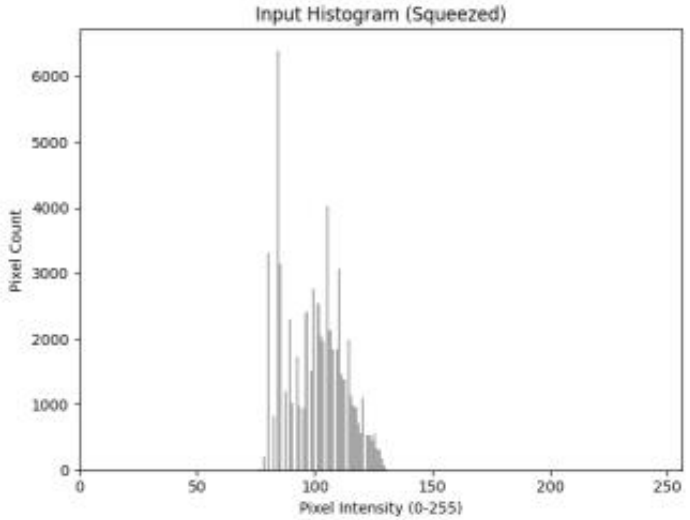
- Write a python code to adjust the contrast of an input image as

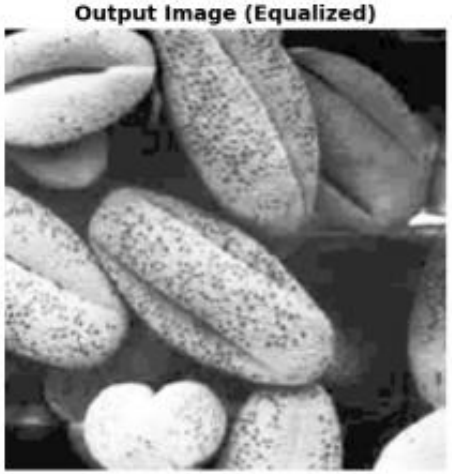


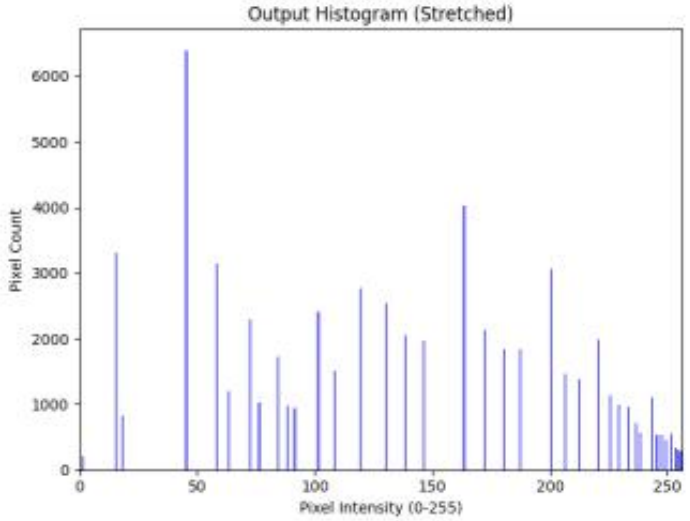
Act 3.4

- Write a python code to adjust the contrast of an input image as



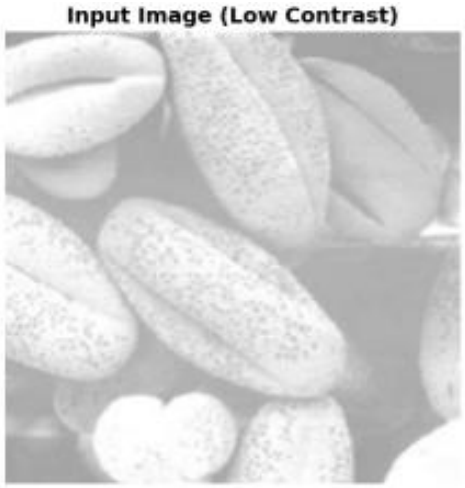


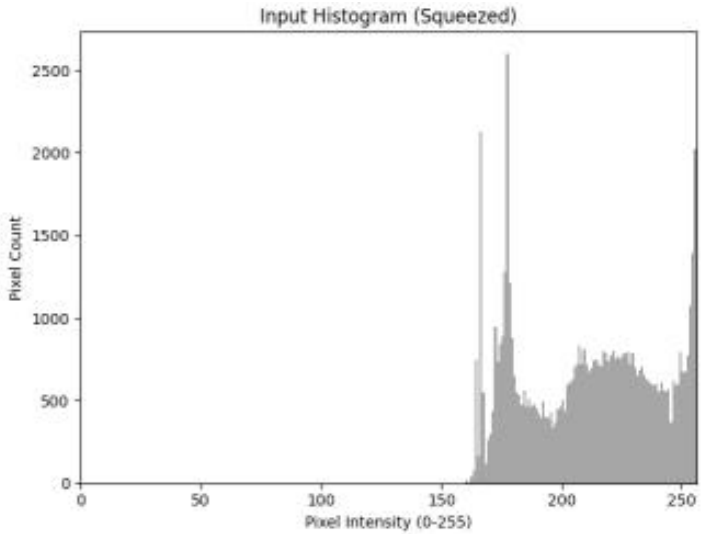


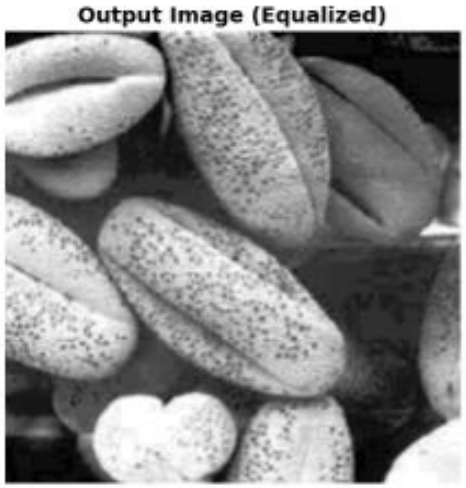


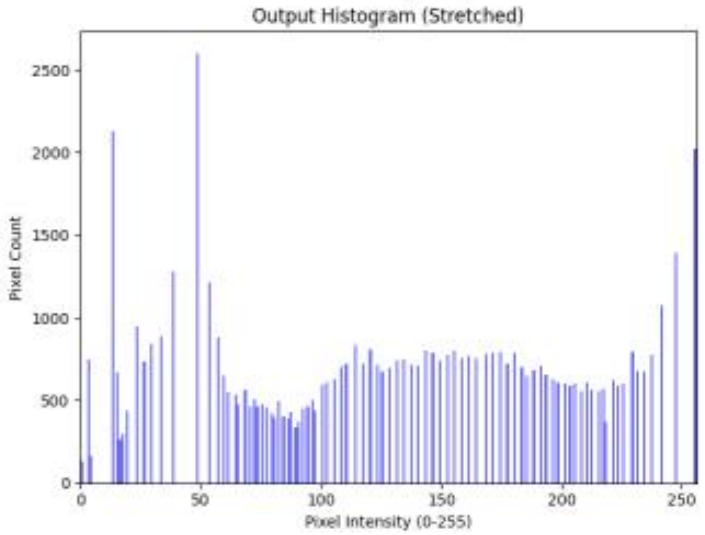
Act 3.4

- Write a python code to adjust the contrast of an input image as



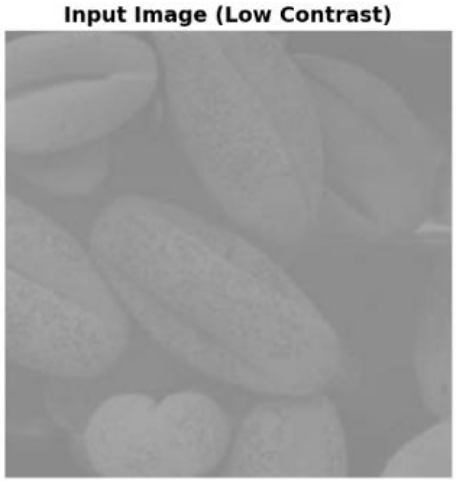


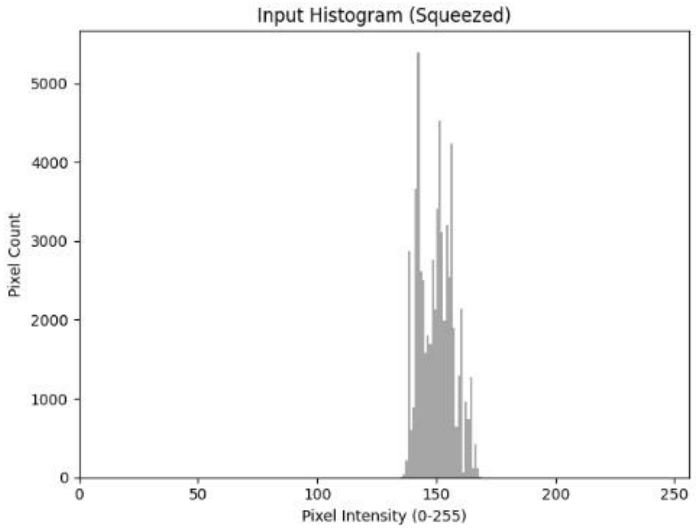




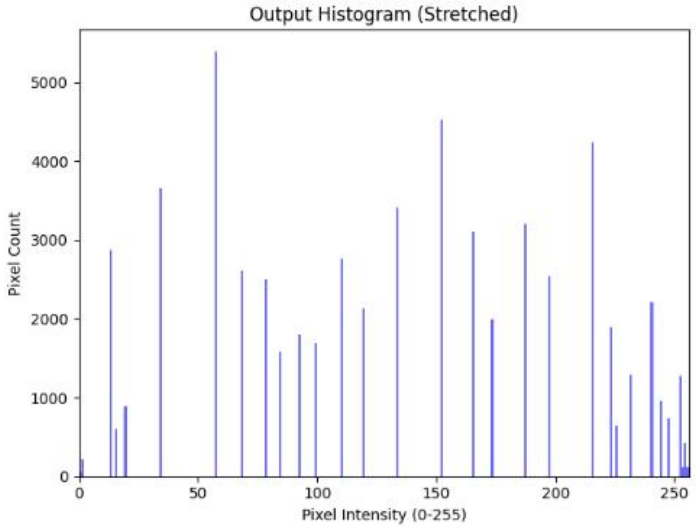
Act 3.4

- Write a python code to adjust the contrast of an input image as





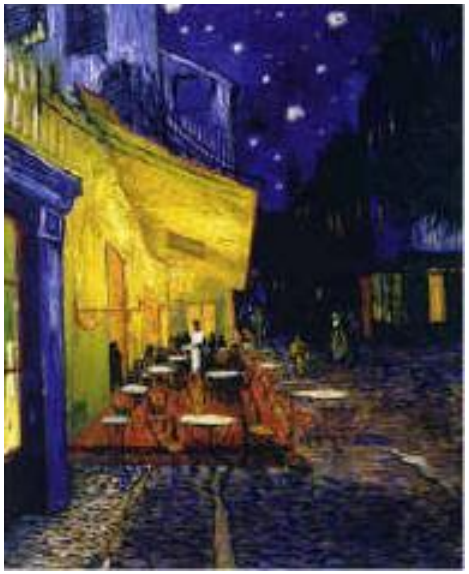




Color Transfer

- Make one image look like another
- Make a synthetic image take on another image's look and feel





Color Transfer Example



Color Transfer Example (cont.)







Color Transfer Example

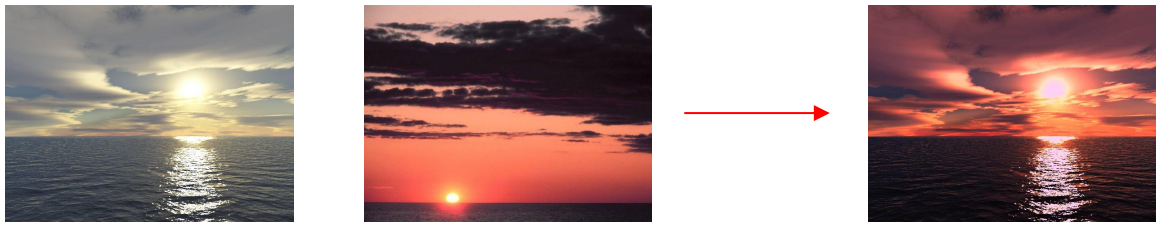






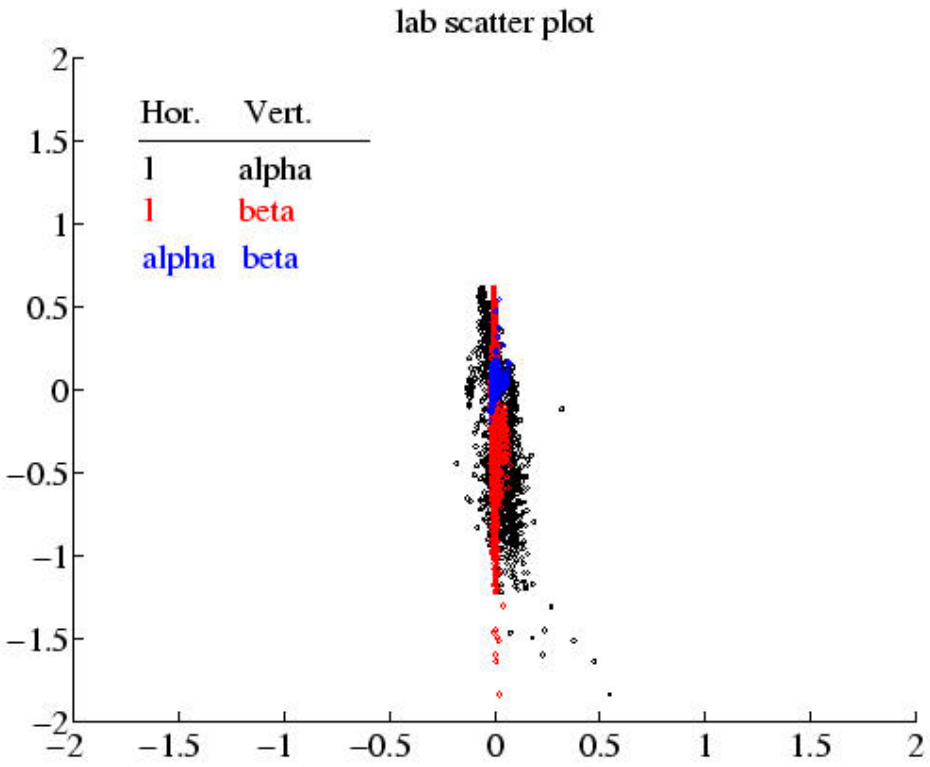
Color Transfer

- 4 steps as follow:
  - Step1: For both images, transfer to new color space, lab
  - Step2: Compute **means** and **standard deviations** along each color channel of both images
  - Step3: **Shift** and **scale** source image to have same statistics as the target image
  - Step4: Transfer the image back to the RGB



Step 1: Transfer to the new color space from RGB to Lab

lαβ Color Space



- Minimizes correlation between channels for many natural scenes

Steps:

- Convert RGB triplets to LMS cone space
- Take logarithm
- Rotate axes

Conversion from RGB to lαβ

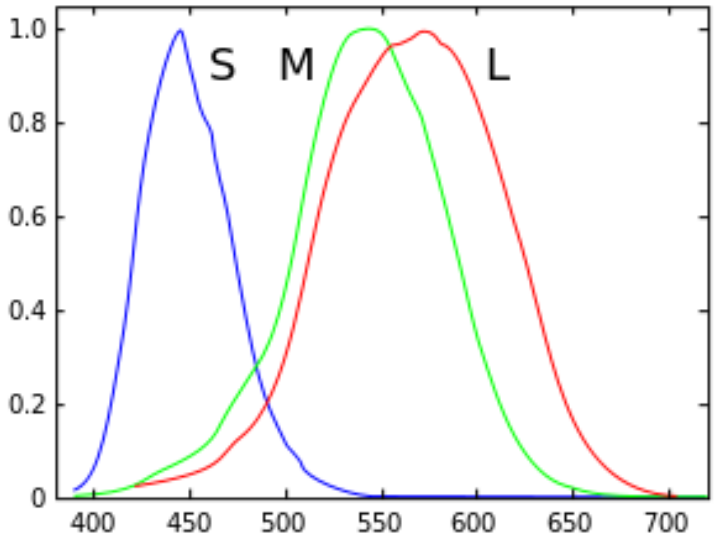
Convert RGB triplets to LMS cone space

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} 0.5141 & 0.3239 & 0.1604 \\ 0.2651 & 0.6702 & 0.0641 \\ 0.0241 & 0.1228 & 0.8444 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

$$\begin{bmatrix} L \\ M \\ S \end{bmatrix} = \begin{bmatrix} 0.3897 & 0.6890 & -0.0787 \\ -0.2298 & 1.1834 & 0.0464 \\ 0.0000 & 0.0000 & 1.0000 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$



$$\begin{bmatrix} L \\ M \\ S \end{bmatrix} = \begin{bmatrix} 0.3811 & 0.5783 & 0.0402 \\ 0.1967 & 0.7244 & 0.0782 \\ 0.0241 & 0.1288 & 0.8444 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$



Conversion from RGB to lαβ

$$\begin{aligned} \boldsymbol{L} &= \log L \\ \boldsymbol{M} &= \log M \\ \boldsymbol{S} &= \log S \end{aligned}$$

Take logarithm

$$\begin{bmatrix} l \\ \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \frac{1}{\sqrt{3}} & 0 & 0 \\ 0 & \frac{1}{\sqrt{6}} & 0 \\ 0 & 0 & \frac{1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & -2 \\ 1 & -1 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{L} \\ \mathbf{M} \\ \mathbf{S} \end{bmatrix}$$

Rotate axes

COLOR_YCrCb2BGR
Python: cv.COLOR_YCrCb2BGR
COLOR_YCrCb2RGB
Python: cv.COLOR_YCrCb2RGB
COLOR_BGR2HSV
Python: cv.COLOR_BGR2HSV
COLOR_RGB2HSV
Python: cv.COLOR_RGB2HSV
COLOR_BGR2Lab
Python: cv.COLOR_BGR2Lab
COLOR_RGB2Lab
Python: cv.COLOR_RGB2Lab

Conversion from RGB to lαβ

$$\begin{bmatrix} l \\ \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \frac{1}{\sqrt{3}} & 0 & 0 \\ 0 & \frac{1}{\sqrt{6}} & 0 \\ 0 & 0 & \frac{1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & -2 \\ 1 & -1 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{L} \\ \mathbf{M} \\ \mathbf{S} \end{bmatrix}$$



$$\begin{aligned} \text{Achromatic} &\propto r+g+b \\ \text{Yellow–blue} &\propto r+g-b \\ \text{Red–green} &\propto r-g \end{aligned}$$

l axis represents an achromatic channel

α channel is chromatic yellow – blue

β channel is chromatic red – green

- The method operates in this lαβ space because de-correlation lets us treat the three color channels separately, simplifying the method.

Steps 2 & 3:
Adjust source image to have same statistics as the target image

Step2: Compute **means** and **standard deviations** along each color channel of both images

Step3: **Shift** and **scale** source image to have same statistics as the target image

Changing Statistics

- Adjusting the **mean and standard** deviations along each of the three axes of lαβ

Changing Statistics

- Compute means and standard deviations for both source and target images for each of the three axes of lαβ.

	Color channels	Means	Standard deviations
Source image I1	L	μ_L^1	σ_L^1
	a	μ_a^1	σ_a^1
	b	μ_b^1	σ_b^1
Target image I2	L	μ_L^2	σ_L^2
	a	μ_a^2	σ_a^2
	b	μ_b^2	σ_b^2

Changing Statistics

- Subtract the mean of the source image from each pixels' value of the source image (l, a, b)
- Scale the pixels' values by factors determined by the respective standard deviations, for the source image
- Add the mean of the target image to the pixels' values of the source image

$$l = (l - \mu_L^1)\frac{\sigma_L^2}{\sigma_L^1} + \mu_L^2$$

$$a = (a - \mu_a^1)\frac{\sigma_a^2}{\sigma_a^1} + \mu_a^2$$

$$b = \left(b - \mu_b^1\right)\frac{\sigma_b^2}{\sigma_b^1} + \mu_b^2$$

Transfer the color from I2 to I1
Change the color of I1 to match I2

Step4: convert the result back to RGB

COLOR_YCrCb2BGR
Python: cv.COLOR_YCrCb2BGR

COLOR_YCrCb2RGB
Python: cv.COLOR_YCrCb2RGB

COLOR_BGR2HSV
Python: cv.COLOR_BGR2HSV

COLOR_RGB2HSV
Python: cv.COLOR_RGB2HSV

COLOR_BGR2Lab
Python: cv.COLOR_BGR2Lab

COLOR_RGB2Lab
Python: cv.COLOR_RGB2Lab

Conversion from lαβ to RGB

$$\begin{bmatrix} \mathbf{L} \\ \mathbf{M} \\ \mathbf{S} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & -1 \\ 1 & -2 & 0 \end{bmatrix} \begin{bmatrix} \frac{\sqrt{3}}{3} & 0 & 0 \\ 0 & \frac{\sqrt{6}}{6} & 0 \\ 0 & 0 & \frac{\sqrt{2}}{2} \end{bmatrix} \begin{bmatrix} l \\ \alpha \\ \beta \end{bmatrix}$$

Invert the log scale then go back to RGB

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 4.4679 & -3.5873 & 0.1193 \\ -1.2186 & 2.3809 & -0.1624 \\ 0.0497 & -0.2439 & 1.2045 \end{bmatrix} \begin{bmatrix} L \\ M \\ S \end{bmatrix}$$

Class Activity 3.5

- Given the l-channels of two lab-images with the size 5x5 (L1 and L2), perform the color transfer from L1 to L2
  - Modify L1 or L2?
- Check the correctness of your answer by checking the mean and standard deviation of the color-transferred image
- Note: Type of the pixel information is 8U
- color transfer from L2 to L1

10	20	30	40	50
20	30	40	50	40
30	40	50	40	30
40	50	40	30	20
50	40	30	20	10

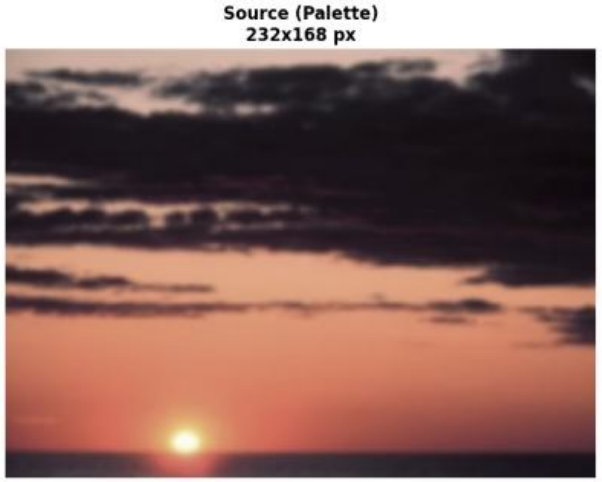
L1

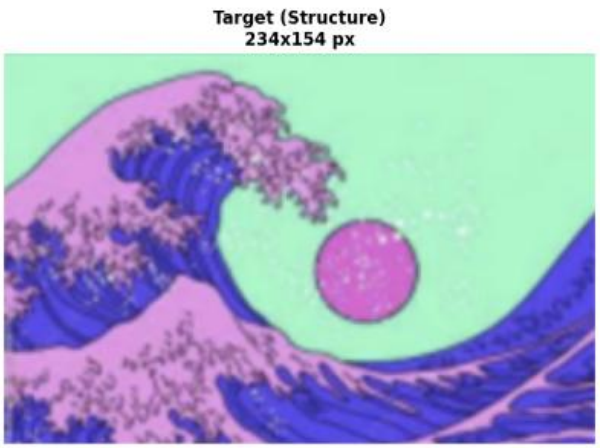
10	10	10	10	10
30	30	30	30	30
50	50	50	50	50
30	30	30	30	30
10	10	10	10	10

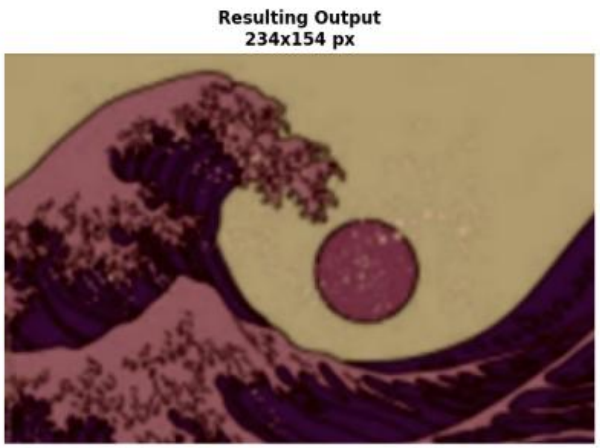
L2

Class Activity 3.6

- Write a python code to perform a color transfer between 2 images







To submit

Individual work (DON'T FORGET to put your name)

Activity 3.1, 3.3, 3.5: Paper-based

Group work (DON'T FORGET to put your members' names)

Activity 3.2

Write a python code to implement a chosen method of morphological operations to get the desired output

Activity 3.4

Write a python code to adjust the contrast of an input image

Activity 3.6

Write a python code to perform a color transfer between 2 images